PROJECT REPORT

14x27

Every secret. One vault. Your control.

Project Report • Product concept, architecture, security model and implementation status

PROJECT FOCUS	Privacy-first password management and digital credential security

PROJECT CHECKPOINT • 10 OCTOBER 2026

Prepared from the current project source package and documented implementation status.

01 / OVERVIEW

Executive summary

14x27 is a privacy-first password manager concept built around a simple promise: keep digital credentials organized, protect sensitive record contents locally, and give the user clear control over saving, generating, importing and backing up secrets. The intended audience includes students, developers, freelancers and everyday users who manage many online accounts and technical credentials.

The project currently has two related workstreams: a web experience being developed in Lovable and a native Android source project. The available Android source package describes a local-first vault built with Kotlin, Jetpack Compose, Room/SQLite, Argon2id and AES-256-GCM. The web workstream is connected to GitHub according to the latest user update; its exact dependency stack and feature parity have not been independently inspected for this report.

STATUS AT A GLANCE

Source and documentation are present, and narrow smoke checks passed for the password generator and CSV parser. The Android project has not been compiled, the JUnit/Android test suite has not run, no APK was generated, and no device-level test or independent security audit has been completed.

Project snapshot

Item	Current definition
Product	14x27 — Secure Password Manager & Digital Credential Vault
Tagline	Every secret. One vault. Your control.
Target users	Students, developers, freelancers and everyday digital users
Android source	Native Kotlin application; application ID: dev.fourteenx27.vault
Storage model	Offline-first local vault with encrypted credential payloads
Current checkpoint	Source stage; build and device verification remain open
Web workstream	Lovable project; GitHub connection confirmed by the user

Document map

This report covers the problem, objectives, system architecture, cryptographic design, feature inventory, verified checks, limitations and the next milestones required before a release claim can be made.

02 / PRODUCT DEFINITION

Problem, objectives and scope

Problem statement

Digital credentials are spread across browsers, websites, development tools and personal notes. Reusing passwords, copying secrets between tools and losing track of old credentials can increase avoidable risk. A useful password manager must combine strong data protection with an understandable workflow; security controls that are confusing or inconvenient are less likely to be used consistently.

Project objectives

- Provide one organized vault for website logins and other digital credentials.
- Encrypt sensitive record content before it is persisted in the native Android database.
- Generate unique passwords using cryptographically secure randomness.
- Offer local password-hygiene findings without presenting a heuristic score as a breach check.
- Support controlled migration from a password-manager CSV export and password-protected encrypted backups.
- Explore Android Autofill to reduce repeated typing while retaining user control.
- Maintain an honest feature and testing status so incomplete security work is not represented as release-ready.

Primary use cases

User need	14x27 response	Boundary
Save a login	Create, view, edit, search and delete an encrypted credential record.	Source present; Android build is unverified.
Create a strong password	Generate a random value with user-selected character classes.	Generated value is not saved unless the user chooses to save it.
Review weak/reused entries	Run a local heuristic hygiene audit.	Not a breach-database check or independent strength certification.
Move existing logins	Import selected entries from a user-exported CSV file.	CSV exports can be plaintext until safely removed.
Back up a vault	Create a password-protected encrypted snapshot.	Export/restore paths need device and malformed-file testing.
Autofill a website	Offer host-matching records through Android Autofill when unlocked.	Coverage depends on form/browser metadata; no locked-vault system unlock flow.

Scope boundaries

The native application is offline-first and the manifest in the source package does not declare INTERNET permission. Remote sync, hosted sharing, account systems, passkeys and a Credential Manager provider are not part of the verified current scope. The Lovable web project is a parallel workstream; the exact web stack and parity with Android must be confirmed from the connected repository before describing them as complete.

03 / ENGINEERING DESIGN

System architecture

The Android source is organized into presentation, domain, data, cryptography, import and Autofill responsibilities. This separation helps keep user-interface code away from raw key handling and makes individual components easier to test. The architecture below describes the contents of the supplied source package; it does not indicate that the project has compiled successfully.

Layer / module	Responsibility	Key elements
Presentation	Screens, navigation and user interactions	Jetpack Compose, Material 3, MainActivity and UI screens
Domain	Credential model and local business logic	CredentialRecord, PasswordGenerator, PasswordAudit
Data	Persistent storage and repository operations	Room/SQLite, VaultDatabase, VaultRepository
Cryptography	Derive keys, wrap the vault key and encrypt records	Bouncy Castle Argon2id, platform AES/GCM, CryptoEngine
Migration / export	Parse CSV and create/restore encrypted snapshots	PasswordCsvImporter, EncryptedBackupCodec
Platform integration	Offer saved credentials to supported Android forms	Android Autofill Framework service

Record creation flow

1. The user creates or unlocks the vault with the master password.
2. A key-encryption key is derived from the password and a random salt using Argon2id.
3. A random 256-bit data-encryption key is generated for vault content and protected by the derived key.
4. A credential payload is serialized and encrypted with AES-256-GCM using a fresh nonce and authenticated record context.
5. Room persists the ciphertext, nonce, cryptographic version and necessary operational metadata; plaintext secret fields are not stored as ordinary credential columns.
6. When the vault is unlocked, matching records are decrypted in memory for display, search, local audit or eligible Autofill requests.

ARCHITECTURE PRINCIPLE

Keep the secret-data path narrow: UI → application logic → cryptographic boundary → local encrypted storage. The current Android manifest has no INTERNET permission, so this source package does not declare a general network path.

Web workstream

The web experience is being built in Lovable and the user reports that GitHub is now connected. A successful Git connection supports version control and change review, but does not itself prove that application logic, hosting, encryption, or web-to-Android synchronization works. For the final repository handoff, record the actual front-end dependencies from package.json and document any backend services or browser-storage choices explicitly.

04 / TRUST FOUNDATION

Security and privacy model

Key hierarchy

The design separates the user-supplied master password from the key-encryption key (KEK) and the random data-encryption key (DEK). The master password is supplied during setup/unlock and is not intended to be persisted. The KEK protects the DEK; the DEK encrypts credential records. This separation makes password changes and record encryption conceptually distinct, though a tested master-password-rotation workflow is not implemented in the current source checkpoint.

Component	Source design	Important note
Password KDF	Argon2id v1.3; initial parameters: 64 MiB memory, 3 iterations, parallelism 1.	Parameters are versioned but have not been benchmarked on target devices.
Salt	16 random bytes stored with vault metadata.	Salt is not secret; it supports per-vault key derivation.
KEK	32-byte key derived from the master password using Argon2id.	Should not be persisted after use.
DEK	Random 32-byte (256-bit) vault data key.	Stored only as an AES-GCM-wrapped value in metadata.
Record encryption	AES-256-GCM, fresh 12-byte nonce and 128-bit authentication tag.	Record AAD binds vault ID, record ID and schema version.
Backup	Separate backup password and Argon2id + AES-GCM encryption.	Requires restore testing against wrong passwords and corrupt files.

Security controls represented in source

- Credential payloads are encrypted before Room persistence; operational IDs and timestamps remain visible.
- The Android manifest disables app backup and does not declare INTERNET permission.
- AES-GCM authentication failures are intended to fail closed rather than return malformed content as a credential.
- The Autofill service provides no dataset when the vault is locked; a host match is required for eligible records.
- The app includes a privacy page, screen-protection flag and content-free activity metadata.

Threats and residual risk

Controls do not eliminate risk from weak master passwords, a compromised operating system, malware running in an unlocked session, memory inspection, copied clipboard content, malicious/oversized import files or implementation defects. Record identifiers, timestamps, ciphertext sizes and key-derivation metadata remain visible in the database. A design document is not proof of secure implementation: compilation, adversarial tests and independent review remain necessary.

NOT IMPLEMENTED AT THIS CHECKPOINT

Biometric unlock with Android Keystore-bound key wrapping, a system-mediated Autofill unlock path, passkeys, remote sync and an independent security audit are not implemented or verified. Do not present these as available product features.

05 / CURRENT SCOPE

Feature inventory and implementation status

“Source present” means relevant code or UI exists in the supplied archive. It does not mean the feature compiled, passed its test suite, or worked on a physical device. This distinction is important for a security-sensitive product.

Feature	Status in available source	What remains
Vault setup, unlock and lock	Source present	Compile; test wrong-password, corrupted metadata and lifecycle paths.
Credential create/read/update/delete	Source present	Run persistence, concurrency and ciphertext-only database tests.
Password generator	Source present; narrow CLI smoke test passed	Run Android/JUnit suite and broader randomized property tests.
Local hygiene audit	Source present; heuristic analysis	Do not call it a breach scan; improve scoring and disclose limitations.
CSV migration	Source present; parser smoke test passed	Fuzz parser, enforce size limits and verify UI/device import behavior.
Encrypted backup/restore	Source present	Test interrupted writes, wrong passwords, tampering, file sizes and restore behavior.
Android Autofill	Service source present	Test browsers/forms on devices; implement and test secure unlock flow.
Collections/developer workspace	Partial / not dedicated workflows	Implement CRUD, tags and environment workflows before claiming completion.
Biometric unlock / Keystore wrapping	Not implemented	Design and test key lifecycle before offering this feature.
APK / production release	Not generated or verified	Obtain a working Android toolchain, compile, test, sign and inspect output.

Technology stack

Native Android: Kotlin; Android SDK; Jetpack Compose; Material 3; Gradle Kotlin DSL; Room/SQLite; Kotlin coroutines; Bouncy Castle Argon2id; platform AES/GCM; Android Autofill Framework; JUnit and AndroidX test dependencies (test execution remains pending). Source configuration specifies minimum SDK 26 and compile/target SDK 36.

Web workstream: Lovable for the web project and GitHub for repository synchronization/version control. Exact framework, styling packages, hosting configuration and any backend integration should be confirmed from the connected repository rather than inferred from the design tool.

06 / EVIDENCE

Verification, risks and release gates

Checks recorded as completed

Check	Recorded result	Interpretation
Password generator smoke test	Passed in a standalone Kotlin CLI run.	Checked requested length and selected character-category inclusion; not an Android build test.
CSV parser smoke test	Passed in a standalone Kotlin CLI run.	Checked quoted comma/escaped quote handling and normalized duplicate-host detection.
Android XML parsing	Passed for all 6 manifest/resource XML files.	Confirms XML well-formedness only.
Manifest INTERNET permission check	No INTERNET declaration found.	A source-manifest inspection, not a network security audit.

Not yet verified

- Gradle dependency resolution and full Kotlin/Compose/Room compilation.
- JUnit crypto tests, Room persistence tests and Android UI/instrumentation tests.
- APK generation, package/signature inspection, installation and live Autofill on a device.
- Backup restoration under adversarial inputs and large-file boundary tests.
- Independent cryptographic review, penetration test or security certification.

Main release risks

Risk	Why it matters	Required evidence before release
Build toolchain unavailable	Compiler/runtime defects may still exist.	Successful Gradle sync, unit tests and debug build.
Unlock and key lifecycle gaps	Biometric flow and configurable lock behavior are incomplete.	Threat-reviewed design and device acceptance tests.
Import/backup attack surface	Malformed or oversized files could cause failures or data loss.	Fuzzing, limits, tamper tests and interrupted-write tests.
Autofill coverage	Origin hints vary by browser and form.	Device/browser test matrix and secure user-selection flow.
No independent review	Cryptographic intent does not establish correct implementation.	Independent review with documented findings and fixes.

RELEASE STATEMENT

At the 10 October 2026 checkpoint, 14x27 is a source-stage project, not a verified release. Do not store high-value production credentials in it until build, test, device and security-review gates have been addressed.

07 / NEXT MILESTONES

Roadmap and conclusion

Milestone	Work items	Exit condition
1. Build baseline	Use Android Studio with compatible JDK/SDK; install Gradle 8.13 and generate the official wrapper; resolve dependencies.	Successful clean Gradle sync and repeatable debug build.
2. Core verification	Run existing unit tests; add Room, key-derivation, wrong-password, tamper and ciphertext-only persistence tests.	All required unit tests pass with synthetic test data.
3. Device acceptance	Test setup/unlock, CRUD, generator, CSV migration, backup/restore, app lock and Autofill on API 26+ devices.	Acceptance matrix completed; limitations documented.
4. Security completion	Implement or explicitly defer biometric key wrapping, secure Autofill unlock and missing workflows.	Threat model updated and no unsupported feature claims.
5. Release readiness	Review third-party dependencies, import size limits, logging, permissions, backup behavior and accessibility; seek independent security review.	Signed artifact, reproducible build record, checksums and review evidence.
6. Web-to-repo handoff	Review GitHub repository, record actual web stack, document setup/hosting/env configuration and keep secrets out of commits.	Clean README, reproducible setup and tracked changes visible in repository.

Conclusion

14x27 is designed to make credential management calmer and more deliberate: one organized vault, secure password generation, local hygiene feedback, controlled migration and encrypted backup. The native source uses a defensible local-first architecture with separated key roles and authenticated encryption. Its current maturity must nevertheless be described accurately: source exists, a small set of narrow checks passed, but full compilation, Android tests, device behavior and independent security review remain outstanding.

The best next step is to turn the source checkpoint into a repeatable build and test baseline before adding higher-risk features. The Lovable web workstream should be documented separately and its dependencies verified from the connected GitHub repository. A successful repository connection is a useful collaboration milestone; it is not a substitute for a working, verified release.

Source materials consulted

14x27 Android source package and its README.md, PROJECT_STATE.md, BUILD_REPORT.md, PROJECT_BLUEPRINT.md, IMPLEMENTATION_PLAN.md, KNOWN_ISSUES.md, CRYPTOGRAPHY_DESIGN.md, SECURITY_THREAT_MODEL.md, DATABASE_SCHEMA.md and TEST_PLAN.md. Project checkpoint date: 10 October 2026.

Web implementation note: Lovable project A14x27s, with GitHub connection reported as completed by the project owner during this conversation. Web-source dependencies and repository contents were not inspected as part of this report.